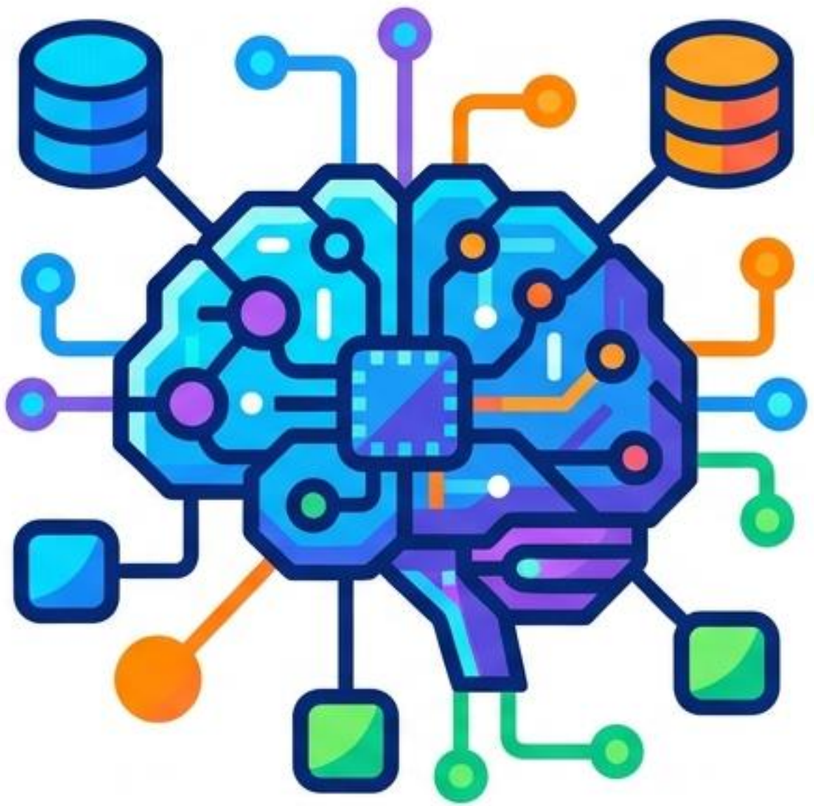
Học Máy (Machine Learning)

Chương 7: Học tổ hợp và Rừng ngẫu nhiên



Giảng viên: TS. Trần Thành Thắng

Đại học Đông Á

August 5, 2026

Gemini Notebook



Nội dung Chương trình

1. Học tổ hợp (Ensemble Learning)
2. Bộ phân loại biểu quyết
3. Túi hóa (Bagging) và Dán nhãn (Pasting)
4. Rừng ngẫu nhiên (Random Forests)
5. Tăng cường (Boosting)
6. Phân lớp xếp chồng (Stacking)
7. Tổng kết Chương 7

Gemini Notebook

Giới thiệu Học tổ hợp

Định nghĩa: Một phương pháp kết hợp các dự đoán của một nhóm các bộ dự đoán (bộ phân loại hoặc bộ hồi quy) để đạt được kết quả dự đoán tốt hơn so với bất kỳ bộ dự đoán riêng lẻ nào.



Một tập hợp (ensemble): Một nhóm các bộ dự đoán.



Học tổ hợp (ensemble learning): Phương pháp học sử dụng nhóm dự đoán này.



Phương pháp tổ hợp (ensemble method): Thuật toán sử dụng kỹ thuật này.

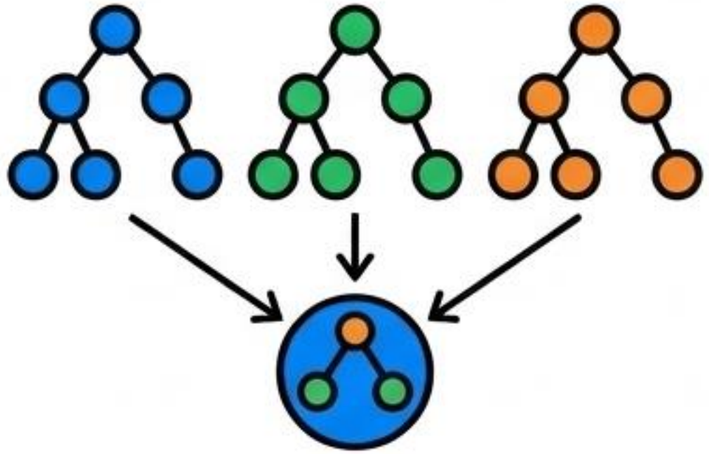
Gemini Notebook

Sự khôn ngoan của đám đông



Giả sử bạn đặt một câu hỏi phức tạp cho hàng nghìn người ngẫu nhiên, sau đó tổng hợp câu trả lời của họ. Trong nhiều trường hợp, câu trả lời tổng hợp này tốt hơn câu trả lời của một chuyên gia.

→ **Sự khôn ngoan của đám đông.**



Ví dụ: Huấn luyện một nhóm các bộ phân loại cây quyết định trên các tập con ngẫu nhiên khác nhau của tập huấn luyện.

→ **Rừng ngẫu nhiên (Random Forest).**

Lưu ý: Thường được áp dụng gần cuối một dự án Machine Learning để đẩy tối đa độ chính xác của hệ thống.

Gemini Notebook

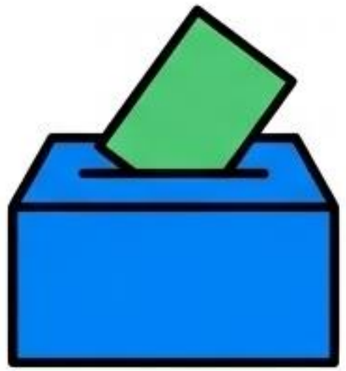
Bộ phân loại biểu quyết (Voting Classifiers)



Khởi tạo: Bắt đầu bằng việc huấn luyện một vài bộ phân loại độc lập trên cùng một tập huấn luyện.



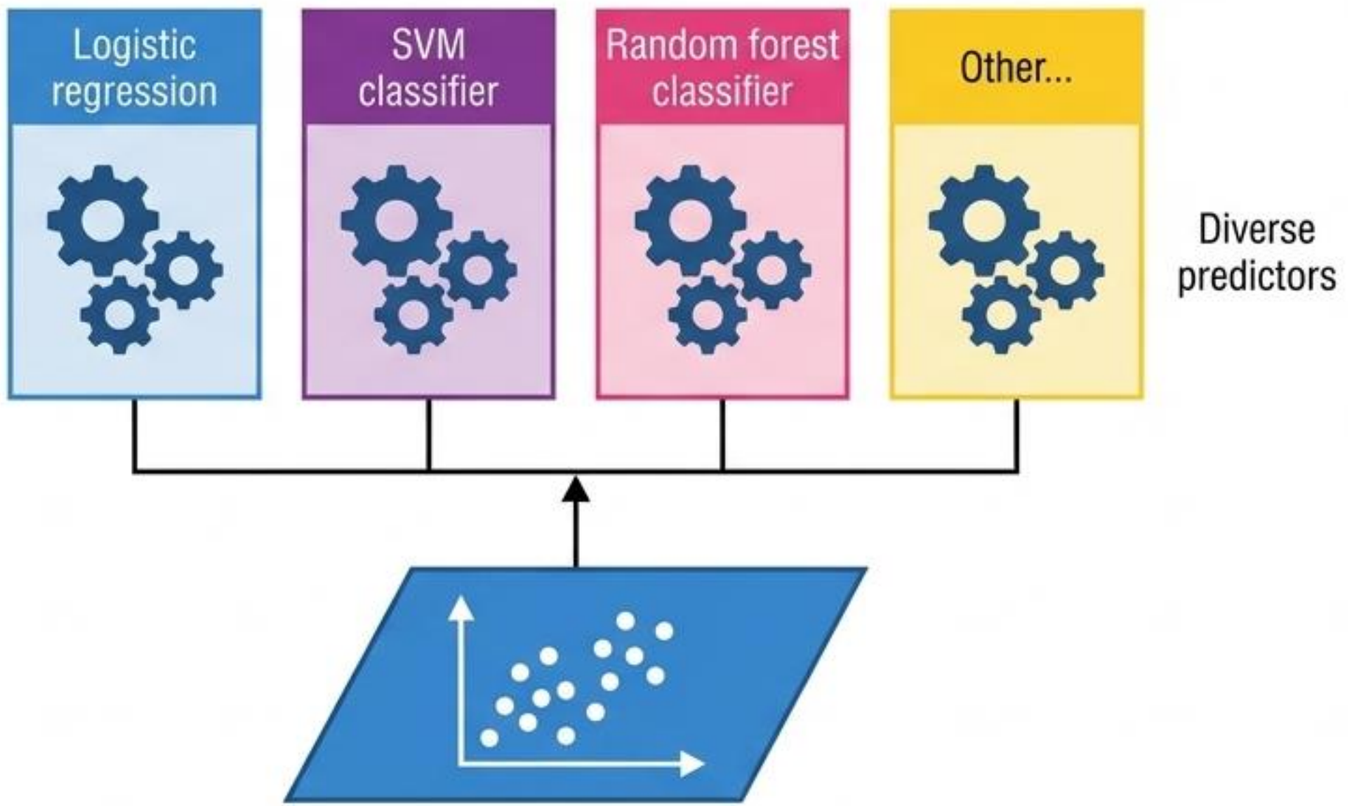
Đa dạng: Mỗi bộ phân loại có thể thuộc các loại khác nhau: SVM, Logistic Regression, Random Forest, K-Nearest Neighbors... (Mỗi bộ phân loại đạt độ chính xác riêng lẻ, ví dụ khoảng 80%.)



Kỹ thuật: Tổng hợp các dự đoán của tất cả các bộ phân loại và đưa ra dự đoán tập hợp.

Gemini Notebook

Huấn luyện các bộ phân loại đa dạng



Hình 7-1. Huấn luyện các bộ phân loại đa dạng

Gemini Notebook

Bộ phân loại bỏ phiếu cứng (Hard Voting)



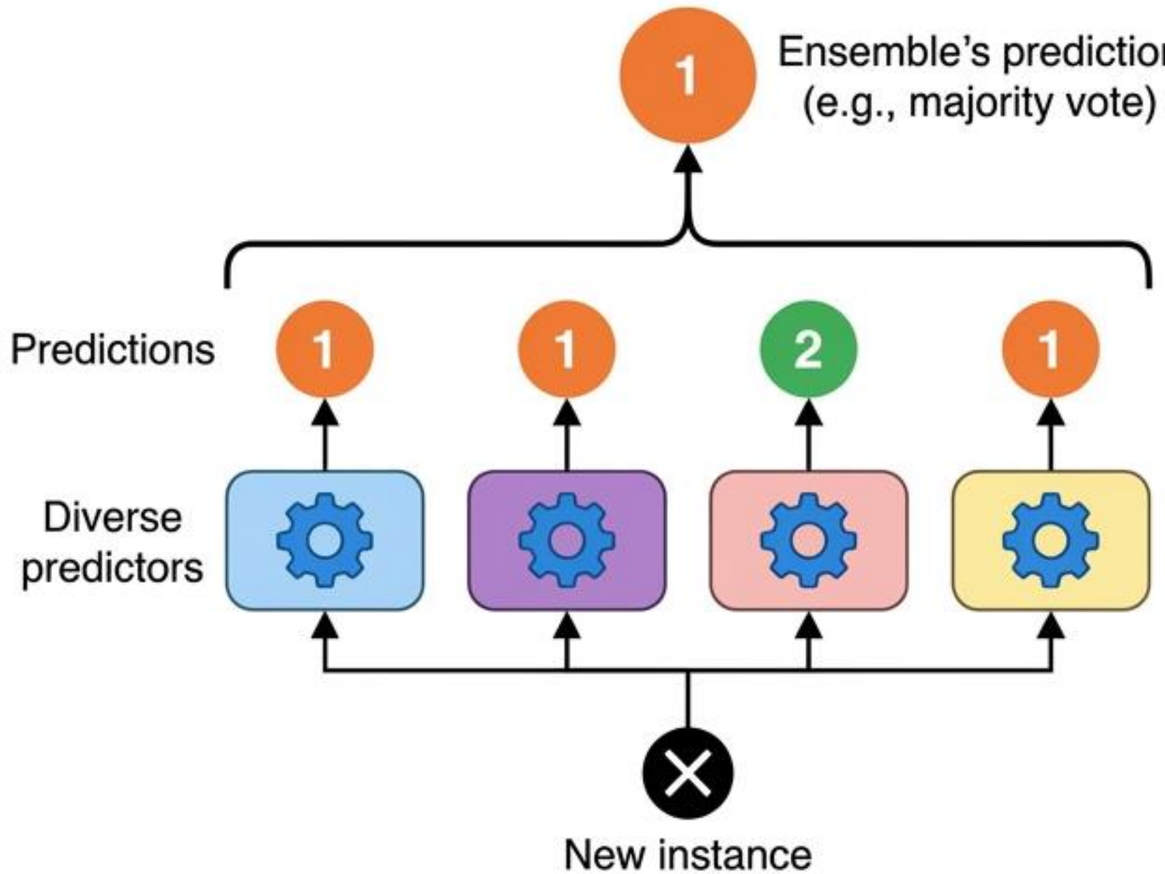
Bỏ phiếu cứng (Hard voting): Dự đoán của tập hợp là lớp nhận được nhiều phiếu bầu đa số nhất từ các bộ phân loại riêng lẻ. Mặc dù đơn giản, bộ phân loại bỏ phiếu cứng thường đạt được độ chính xác cao hơn bộ phân loại tốt nhất trong tập hợp.



Bộ học yếu vs Bộ học mạnh: Ngay cả khi mỗi bộ phân loại chỉ là một bộ học yếu (weak learner - chỉ tốt hơn đoán ngẫu nhiên một chút), tập hợp vẫn có thể trở thành một bộ học mạnh (strong learner) nếu có đủ số lượng bộ học đa dạng.

Gemini Notebook

Dự đoán bỏ phiếu cứng



Dự đoán bỏ phiếu cứng

Gemini Notebook

Nguyên lý đằng sau bộ phân loại biểu quyết

Làm sao để các bộ học yếu kết hợp lại thành bộ học mạnh?



Phép tương tự: Tung đồng xu. Tung một đồng xu hơi lệch (51% sấp, 49% ngửa).



1.000 lần: Tỷ lệ mặt sấp sẽ tiến gần **51%**. Xác suất để đa số (trên 500 lần) là mặt sấp lên đến ~ **75%**.



10.000 lần: Xác suất đạt đa số mặt sấp tăng lên **> 97%.**

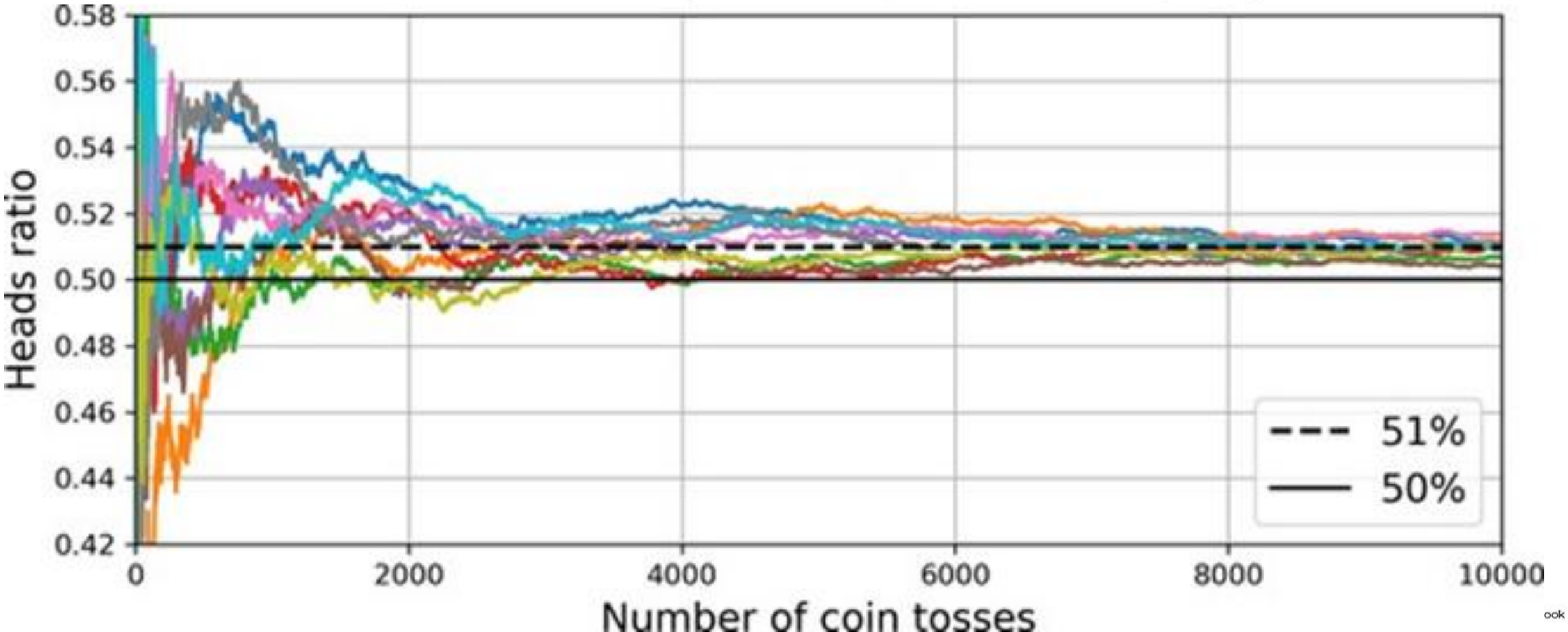


Đây là **ứng dụng của Luật số lớn (Law of Large Numbers).**

Gemini Notebook

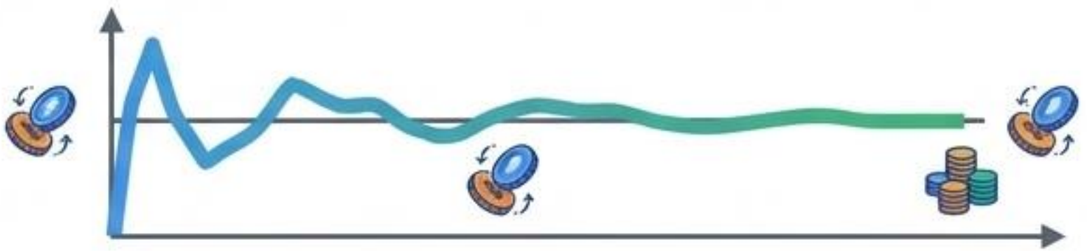
Luật số lớn

Hình 7-3. Luật số lớn - Tỷ lệ mặt sấp khi số lần tung tăng



Luật số lớn

Hình 7-3. Luật số lớn - Tỷ lệ mặt sấp khi số lần tung tăng



Lưu ý quan trọng:

Tập hợp chỉ hoạt động hoàn hảo khi các bộ phân loại hoàn toàn độc lập, mắc lỗi không tương quan.

Do cùng dùng một tập dữ liệu, chúng thường tương quan → làm giảm độ chính xác tập hợp so với kỳ vọng lý thuyết.

Gemini Notebook

Mã nguồn: Bộ phân loại bỏ phiếu cứng (Phần 1)

Sử dụng **VotingClassifier** trong Scikit-Learn trên tập dữ liệu **moons**:

```python
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier , VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

Gemini Notebook

Mã nguồn: Bộ phân loại bỏ phiếu cứng (Phần 2)

Định nghĩa và huấn luyện tập hợp:

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)

# Kiem tra ket qua:
voting_clf.score(X_test, y_test) # Output: 0.912
```

✅ **Kết quả:** Độ chính xác 91.2%, cao hơn độ chính xác của Logistic (86.4%), Random Forest (89.6%) và SVC (89.6%).

Gemini Notebook

Bỏ phiếu mềm (Soft Voting)

Nếu tất cả các bộ phân loại có khả năng ước tính **xác suất lớp** (có phương thức **predict_proba()),** tập hợp có thể đưa ra dự đoán dựa trên trung bình xác suất của từng lớp trên tất cả các bộ phân loại.

Bỏ phiếu mềm (Soft voting): Chọn lớp có xác suất trung bình cao nhất.

Ưu điểm: *Thường đạt hiệu suất cao hơn bỏ phiếu cứng vì nó ưu tiên các phiếu bầu có độ tin cậy cao.*



Gemini Notebook

Mã nguồn: Bỏ phiếu mềm

Đặt siêu tham số **voting="soft"** và đảm bảo các mô hình (như SVC) có tính xác suất (**probability=True**):

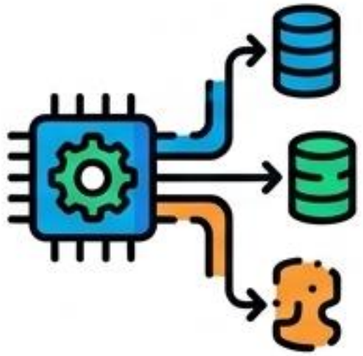
```
voting_clf.voting = "soft"
voting_clf.named_estimators["svc"].probability = True
voting_clf.fit(X_train, y_train)
voting_clf.score(X_test, y_test)
# Output: 0.92
```



Kết quả: Độ chính xác tăng lên 92% chỉ nhờ sử dụng bỏ phiếu mềm.

Gemini Notebook

Túi hóa (Bagging) và Dán nhãn (Pasting)



Thay vì dùng nhiều thuật toán khác nhau, một cách tiếp cận khác là sử dụng cùng một thuật toán cho mỗi bộ dự đoán nhưng huấn luyện trên các tập con ngẫu nhiên khác nhau của tập huấn luyện.

Túi hóa (Bagging - Bootstrap Aggregating)	**Dán nhãn (Pasting)**
Lấy mẫu ngẫu nhiên có hoàn lại (có thay thế). Một mẫu có thể được chọn nhiều lần cho cùng một bộ dự đoán.	Lấy mẫu ngẫu nhiên không hoàn lại (không thay thế). Một mẫu chỉ được chọn nhiều nhất một lần cho mỗi bộ dự đoán.

Gemini Notebook

Đặc điểm của Bagging và Pasting



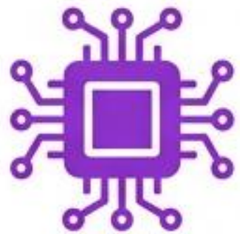
Hàm tổng hợp

Chế độ thống kê (đối với phân loại) hoặc giá trị trung bình (đối với hồi quy).



Bias / Variance

Bộ dự đoán lẻ: độ lệch (bias) cao hơn.
Kết hợp lại: độ lệch tương tự nhưng phương sai (variance) thấp hơn rất nhiều.

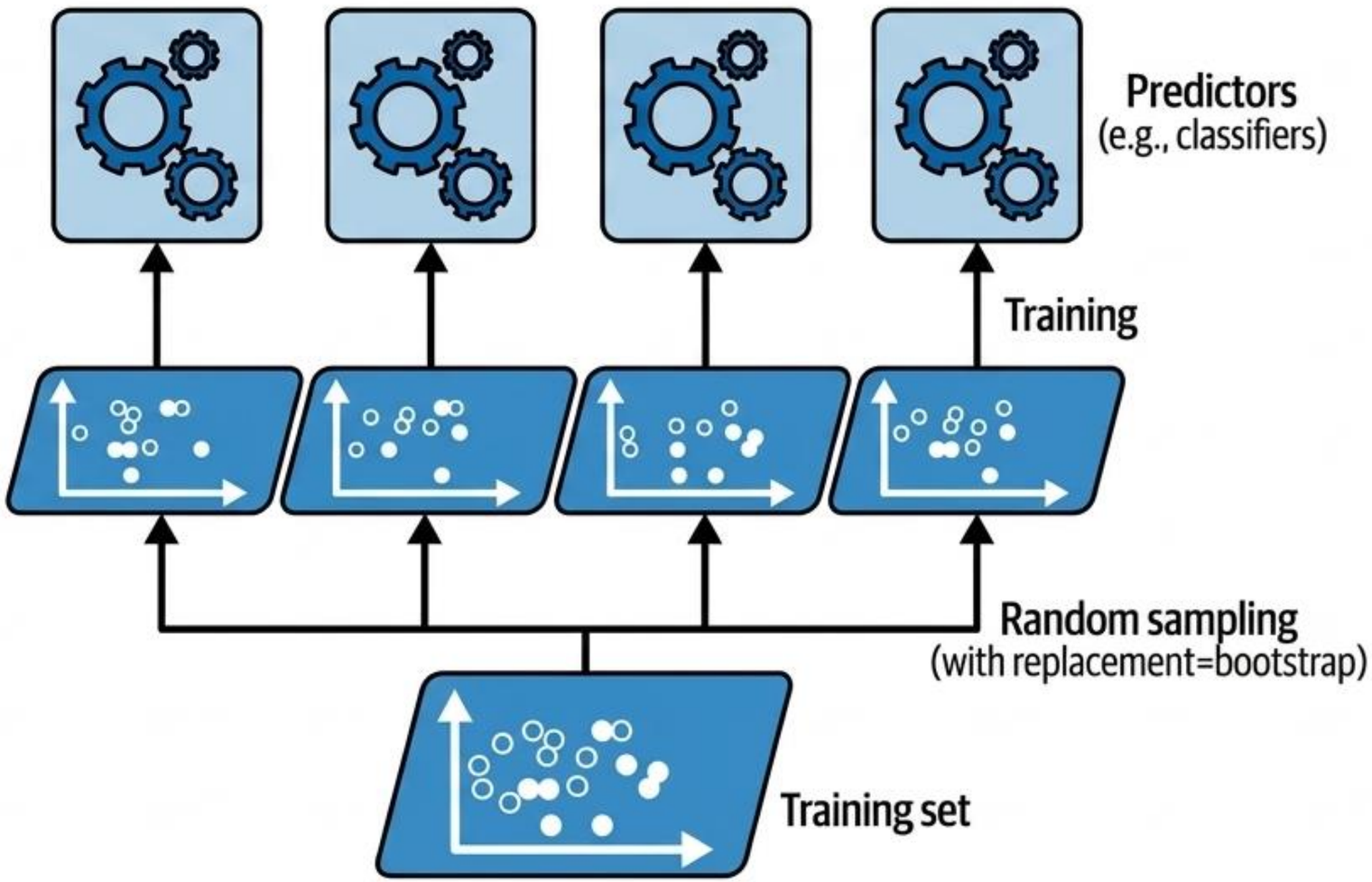


Khả năng mở rộng

Mở rộng tốt. Huấn luyện song song dễ dàng trên đa lõi CPU hoặc cụm máy chủ.

Gemini Notebook

Quá trình lấy mẫu Bagging/Pasting



Hình 7-4. Túi hóa và Dán nhãn trên các mẫu ngẫu nhiên.

Gemini Notebook

Bagging và Pasting trong Scikit-Learn

Scikit-Learn cung cấp lớp `**BaggingClassifier**` (hoặc `**BaggingRegressor**`).



n_estimators

Số lượng bộ dự đoán (ví dụ: 500 cây).



max_samples

Kích thước tập mẫu ngẫu nhiên cho mỗi bộ.



bootstrap=True

Sử dụng Bagging (False cho Pasting).



n_jobs=-1

Sử dụng tất cả các lõi CPU để tăng tốc.

Gemini Notebook

Mã nguồn: BaggingClassifier



Huấn luyện tập hợp gồm 500 cây quyết định bằng phương pháp Bagging.

```python
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(),
    n_estimators=500,
    max_samples=100,
    bootstrap=True, # Doi thanh False de dung Pasting
    n_jobs=-1,
    random_state=42
)
bag_clf.fit(X_train, y_train)
```

Gemini Notebook

Phân tích kết quả của Bagging



Đa dạng hơn trong các tập con huấn luyện (độ lệch nhỉnh hơn một chút so với Pasting).



Sự đa dạng giúp các bộ dự đoán ít tương quan hơn.

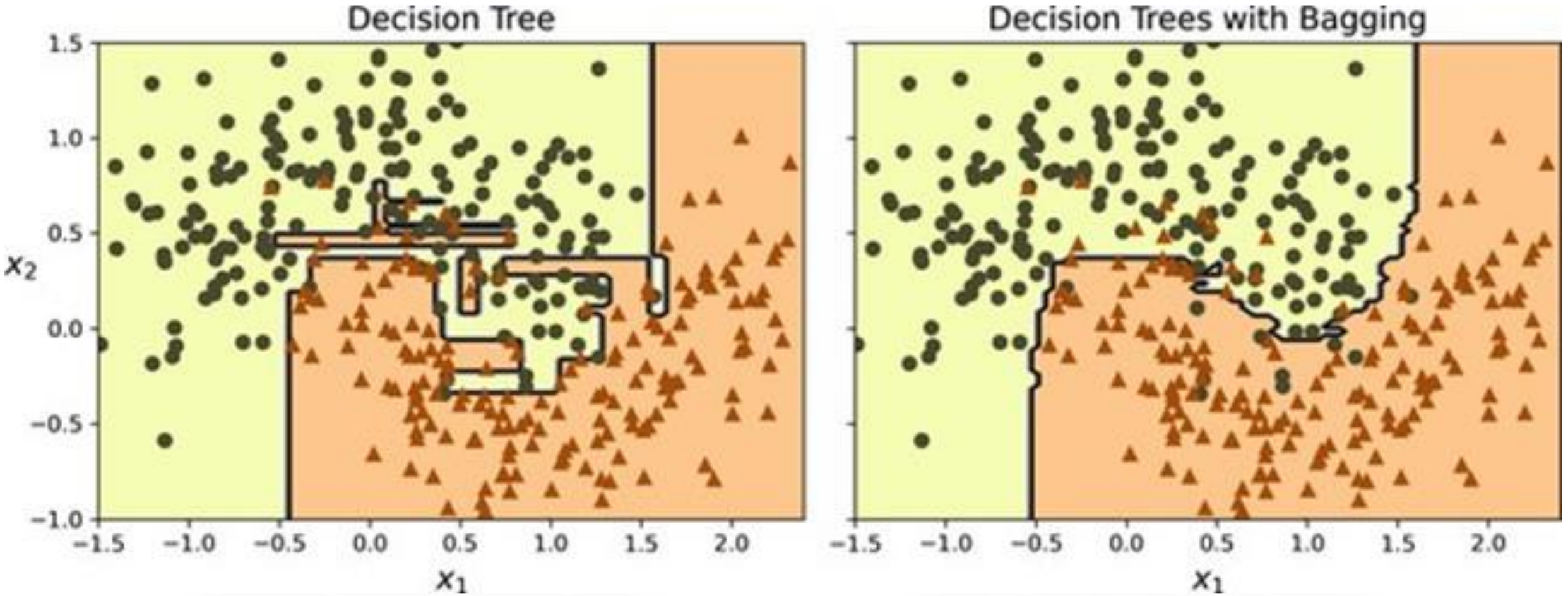


Phương sai của tập hợp được giảm mạnh hơn.

Nhìn chung, Bagging thường mang lại mô hình tổng quát hóa tốt hơn và là lựa chọn ưu tiên mặc định.

Gemini Notebook

So sánh Cây đơn lẻ và Tập hợp túi hóa



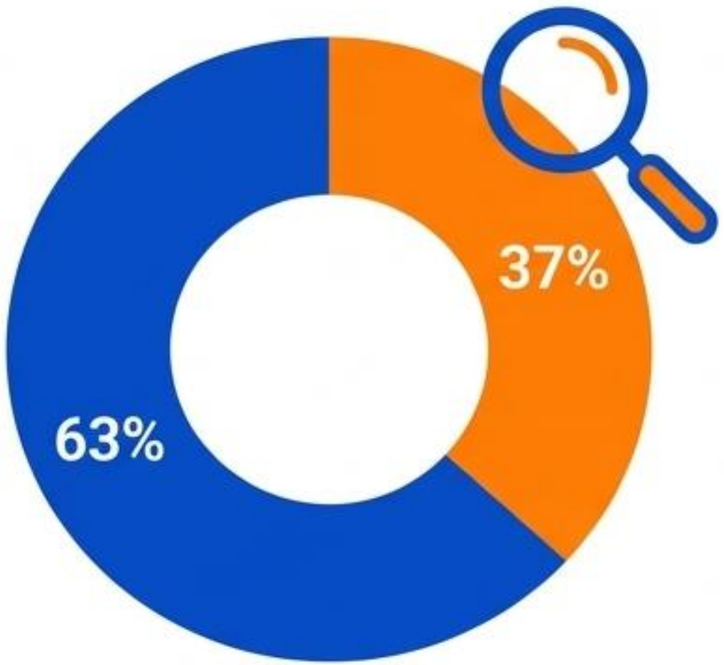
Cây quyết định đơn lẻ **Tập hợp Bagging 500 cây**

Hình 7-5. Trực quan hóa sự khác biệt giữa mô hình dự đoán đơn lẻ và sức mạnh tổng hợp của Bagging.

Gemini Notebook

Đánh giá ngoài mẫu (Out-of-Bag Evaluation)

Việc lấy mẫu có hoàn lại (bootstrap) khiến một số trường hợp được lấy nhiều lần, số khác thì không.



63%

Số trường hợp trung bình mỗi bộ dự đoán lấy.

37%

Các trường hợp ngoài mẫu **(Out-of-Bag - OOB) không bao giờ được nhìn thấy.**

Có thể dùng 37% OOB này để đánh giá tập hợp mà không cần tập xác thực riêng biệt.

Gemini Notebook

Mã nguồn: Đánh giá OOB

Yêu cầu Scikit-Learn tính toán điểm OOB tự động bằng cách bật tham số `oob_score=True`.

```python
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    oob_score=True, n_jobs=-1, random_state=42
)
bag_clf.fit(X_train, y_train)

# Xem diem OOB
print(bag_clf.oob_score_)  # Output: 0.896

# Kiem tra tren tap Test thuc te
from sklearn.metrics import accuracy_score
y_pred = bag_clf.predict(X_test)
print(accuracy_score(y_test, y_pred))  # Output: 0.92
```

Gemini Notebook

Random Patches và Random Subspaces

Lớp `BaggingClassifier` hỗ trợ lấy mẫu theo đặc trưng (features) thông qua `max_features`và `bootstrap_features`.



Random Patches

Lấy mẫu cả trường hợp huấn luyện và đặc trưng.

Rất hiệu quả cho dữ liệu nhiều chiều (chẳng hạn như hình ảnh).



Random Subspaces

Giữ toàn bộ trường hợp huấn luyện nhưng chỉ lấy mẫu ngẫu nhiên đặc trưng.

Đổi một chút độ lệch cao hơn để lấy phương sai thấp hơn.

Gemini Notebook

Rừng ngẫu nhiên (Random Forests)

Tập hợp các cây quyết định, thường được huấn luyện bằng Bagging (với `max_samples` bằng toàn bộ kích thước tập).

Đưa thêm sự ngẫu nhiên: Thay vì tìm đặc trưng tốt nhất trong toàn bộ, thuật toán tìm đặc trưng tốt nhất trong một tập con ngẫu nhiên (mặc định là $\sqrt{n}$ đặc trưng).



Đa dạng cây lớn hơn → đổi độ lệch cao hơn lấy phương sai thấp hơn → mô hình tổng quát hóa tốt.

Gemini Notebook

Mã nguồn: RandomForestClassifier



Sử dụng lớp `RandomForestClassifier` tiện lợi và tối ưu hơn so với dùng `BaggingClassifier` lồng `DecisionTreeClassifier`.

```python
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(
    n_estimators=500,
    max_leaf_nodes=16,
    n_jobs=-1,
    random_state=42
)
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

Gemini Notebook

So sánh Bagging và Random Forest



Mã lệnh `BaggingClassifier` sau đây gần tương đương với `RandomForestClassifier` trước đó:

```python
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500,
    n_jobs=-1,
    random_state=42
)

```

Lớp RandomForestClassifier được tối ưu hiệu năng tốt hơn nhiều cho cây quyết định.

Gemini Notebook

Cây ngẫu nhiên cực đại (Extra-Trees)

Sử dụng các ngưỡng ngẫu nhiên cho mỗi đặc trưng thay vì tìm ngưỡng chia cắt tốt nhất có thể. Tập hợp này được gọi là Extra-Trees (Extremely Randomized Trees).

Lợi ích:

- Tăng tốc độ huấn luyện đáng kể do không phải tốn kém thời gian tìm ngưỡng tốt nhất.
- Đổi thêm một chút độ lệch lấy phương sai thấp hơn nữa.



Gemini Notebook

Cài đặt Extra-Trees trong Scikit-Learn



Trong `DecisionTreeClassifier`, đặt tham số `splitter="random"`, hoặc sử dụng tập hợp trực tiếp bằng `ExtraTreesClassifier`:

```python
from sklearn.ensemble import ExtraTreesClassifier

extra_clf = ExtraTreesClassifier(
    n_estimators=500,
    max_leaf_nodes=16,
    n_jobs=-1,
    random_state=42
)
# API giong het voi RandomForestClassifier

```

Gemini Notebook

Tầm quan trọng của đặc trưng (Feature Importances)

Rừng ngẫu nhiên rất tuyệt vời để đo lường tầm quan trọng tương đối của từng đặc trưng.



Scikit-Learn tính toán mức độ mà các nút cây sử dụng một đặc trưng để làm giảm **độ không tinh khiết** (impurity).

Tính trung bình trên toàn bộ rừng.

Kết quả được **chuẩn hóa**: tổng độ quan trọng của tất cả các đặc trưng bằng 1.

Thuộc tính chứa kết quả trong Scikit-Learn: `feature_importances_`

Gemini Notebook



Máy học: Tăng cường (Boosting)

Chương 7 (Phần 3)

Bài giảng Giới thiệu Bậc Đại học

Gemini Notebook

Mã nguồn: Tính toán Feature Importances

```
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
rnd_clf = RandomForestClassifier(n_es
timators=500, random_state=42)
rnd_clf.fit(iris.data, iris.target)

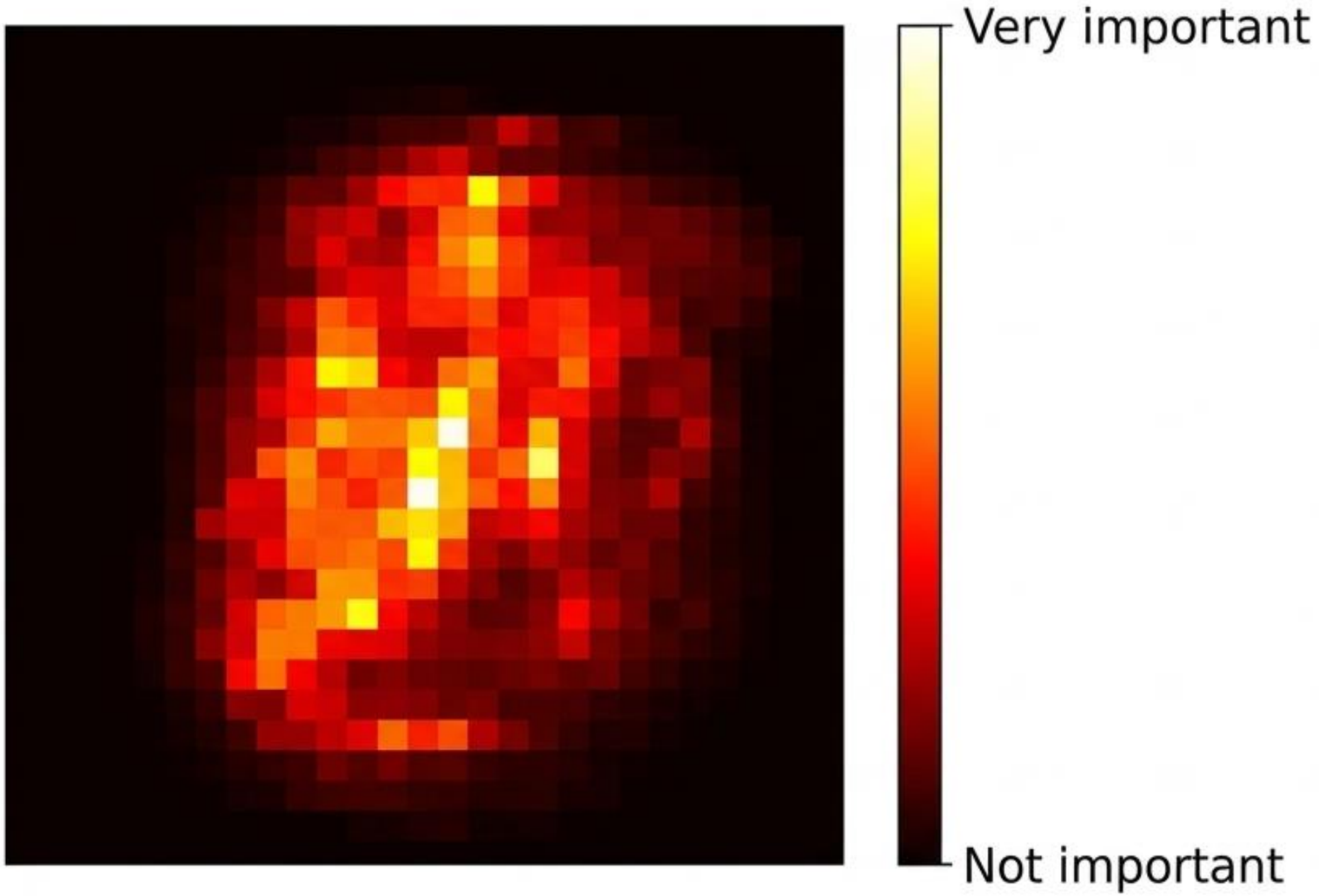
for score, name in
zip(rnd_clf.feature_importances_,
iris.data.columns):
    print(round(score, 2), name)
```



```
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm) <-- Quan trọng nhất
0.42 petal width (cm) <-- Quan trọng thứ hai
```

Gemini Notebook

Tầm quan trọng pixel MNIST



Hình 7-6. Tầm quan trọng của pixel trên ảnh MNIST

Gemini Notebook

Khái niệm Tăng cường (Boosting)



Boosting (Tăng cường giả thuyết) biến các bộ học yếu thành bộ học mạnh.

Huấn luyện các bộ dự đoán theo trình tự, mỗi bộ sau sẽ cố gắng sửa chữa lỗi của bộ tiền nhiệm.

1 AdaBoost (Adaptive Boosting)

2 Gradient Boosting

Gemini Notebook

Trực giác về AdaBoost (Adaptive Boosting)

Tập trung nhiều hơn vào các trường hợp huấn luyện bị phân loại sai bởi bộ tiền nhiệm.



Huấn luyện mô hình cơ sở (thường là Cây quyết định một nút) và dự đoán.



Tăng trọng số tương đối của các trường hợp bị sai.

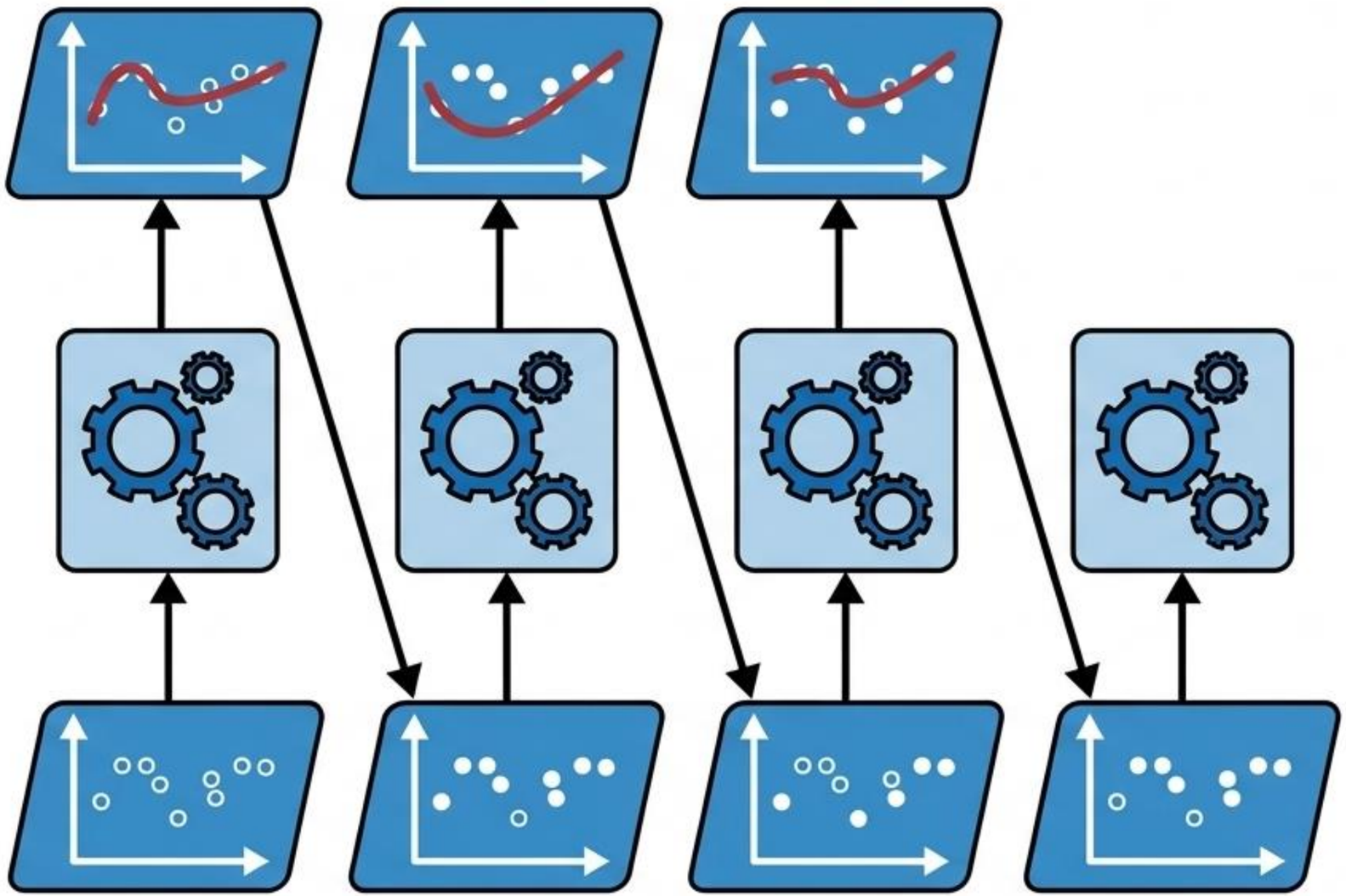


Huấn luyện mô hình thứ 2 dựa trên trọng số mới cập nhật, dự đoán, tăng trọng số, v.v.

Kỹ thuật này khá giống **Gradient Descent**, nhưng thay vì chỉnh tham số cho một mô hình, nó thêm **mô hình mới** vào tập hợp.

Gemini Notebook

Cập nhật trọng số trong AdaBoost



Hình 7-7. *Huấn luyện AdaBoost tuần tự với cập nhật trọng số*

Gemini Notebook

Phân tích trọng số trong SVM (Dữ liệu moons)



Learning Rate Lớn

Bộ dự đoán thứ 2 sửa rất mạnh các lỗi của bộ đầu tiên.



Learning Rate bằng Một nửa

Các SVM sửa đổi một cách cẩn thận và dần dần hơn.

Gemini Notebook

Toán học AdaBoost: Tỷ lệ lỗi & Trọng số

$$r_j = \frac{\sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

Tỷ lệ lỗi có trọng số của bộ thứ j (r_j).

Trọng số ban đầu: $w^{(i)} = 1/m$.

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

Trọng số của bộ dự đoán thứ j (α_j).



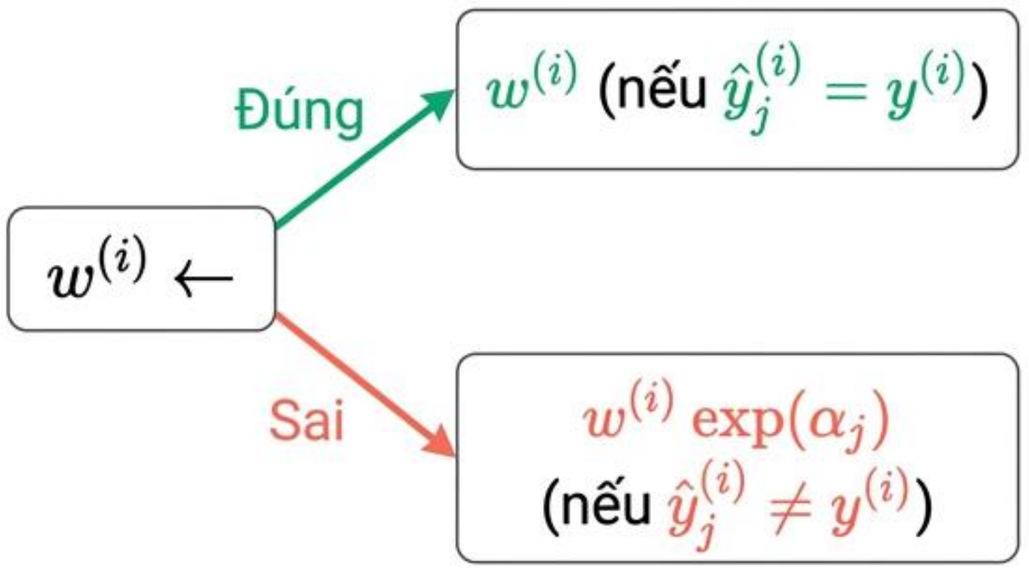
Bộ càng chính xác → α_j càng cao.
Đoán bừa → $\alpha_j \approx 0$.

η là learning rate (mặc định 1).

Gemini Notebook

Toán học AdaBoost: Cập nhật & Dự đoán

Quy tắc cập nhật trọng số trường hợp ($w^{(i)}$)



**Sau đó, tất cả trọng số được chuẩn hóa (chia cho $\sum w^{(i)}$).*



Dự đoán cuối cùng

$$\hat{y}(x) = \underset{k}{\operatorname{argmax}} \sum_{j=1, \hat{y}_j(x)=k}^{N} \alpha_j$$

Gemini Notebook

Mã nguồn: AdaBoostClassifier

Scikit-Learn sử dụng biến thể đa lớp tên là SAMME (hoặc SAMME.R để tận dụng xác suất).

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), \
    n_estimators=30,
    learning_rate=0.5,
    random_state=42
)
ada_clf.fit(X_train, y_train)
```

Cây 1 nút
(decision stump)

Gemini Notebook

Gradient Boosting: Khác biệt cốt lõi

Tương tự AdaBoost: huấn luyện bộ sau sửa lỗi của bộ trước.



AdaBoost Nhắc lại

Thay đổi trọng số trường hợp bị sai.



Gradient Boosting

Khớp bộ dự đoán mới với các **lỗi còn lại (residual errors)** do ưr do bộ dự đoán trước tạo ra.

Gradient Tree Boosting (GBRT): Sử dụng Cây quyết định làm bộ cơ sở.

Gemini Notebook

Gradient Tree Boosting Thủ công (Phần 1)

Tạo dữ liệu và huấn luyện cây đầu tiên



```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100)

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

Gemini Notebook

Gradient Tree Boosting Thủ công (Phần 2)

Huấn luyện trên phần dư (residuals) và kết hợp

```
# Phân dư 1
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)

# Phân dư 2
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)

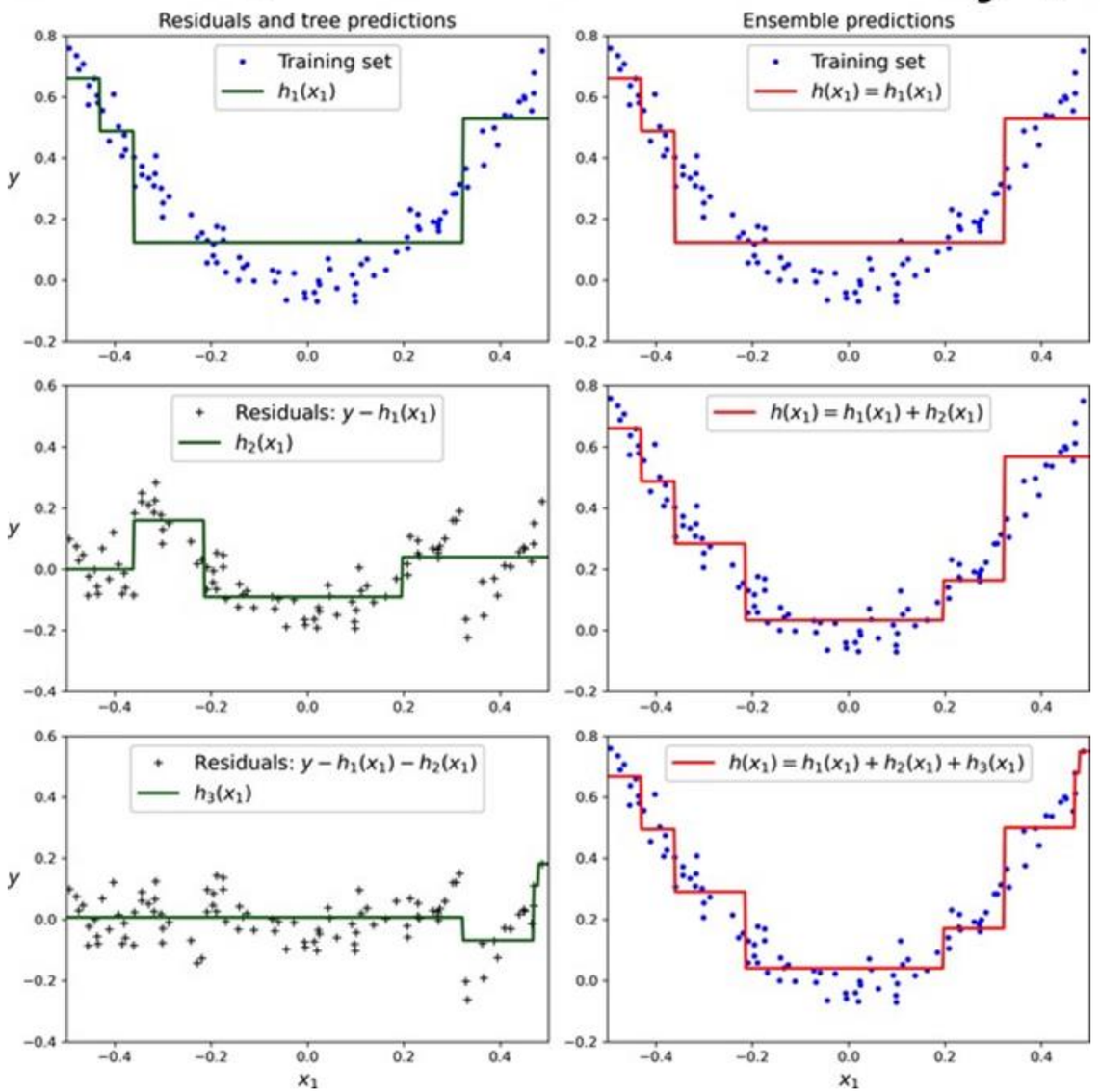
# Dự đoán mới bằng tổng các cây:
X_new = np.array([[-0.4], [0.], [0.5]])
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```





Gemini Notebook

Trực quan hóa: Quá trình huấn luyện GBRT



GBRT trong Scikit-Learn & Vấn đề Shrinkage



```
from sklearn.ensemble import GradientBoostingRegressor
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                 learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

learning_rate: Kiểm soát mức độ đóng góp của từng cây.
Giá trị thấp -> Cần nhiều cây hơn (Shrinkage - Phương pháp chính quy hóa).



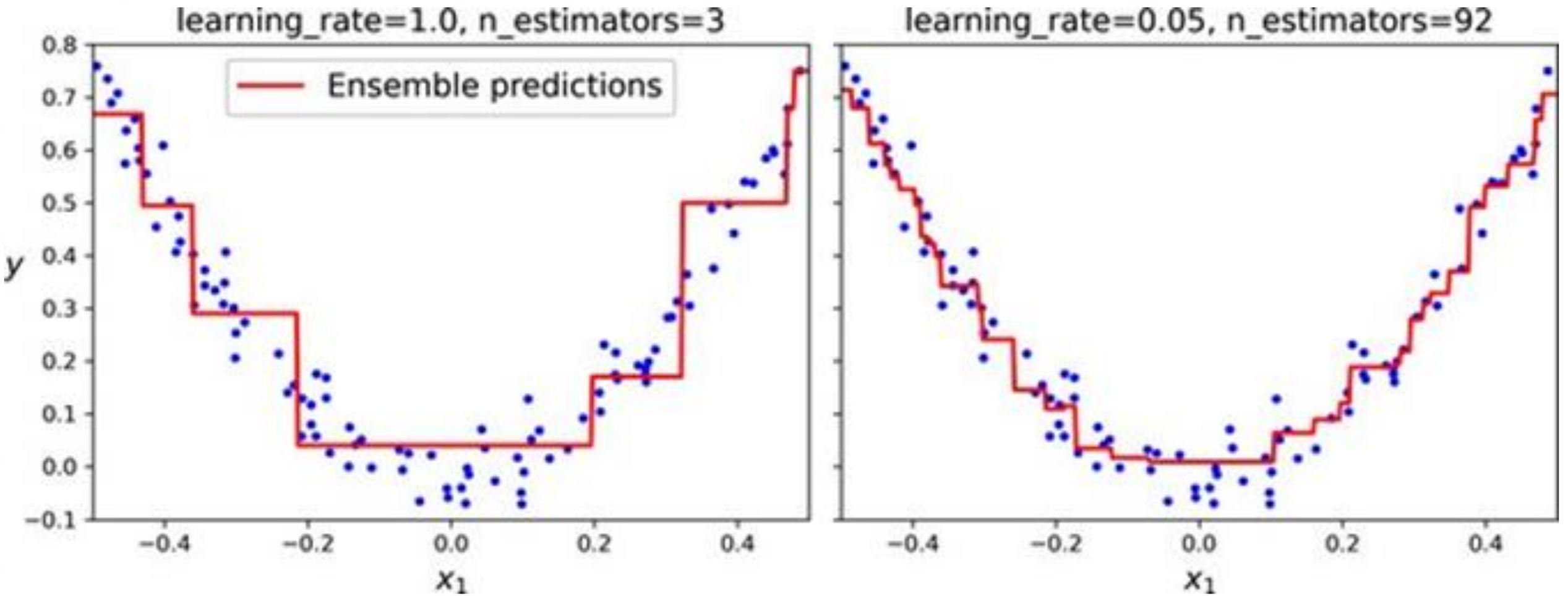
Số lượng cây n_estimators quá Ít -> Dưới khớp (Underfitting).



Số lượng cây n_estimators quá Nhiều -> Quá khớp (Overfitting).

Gemini Notebook

Các tập hợp GBRT không đủ và vừa đủ cây



Hình 7-10. Dưới khớp (trái) vs. Vừa vặn (phải)

Gemini Notebook

Dừng sớm (Early Stopping) trong GBRT



Tránh phải dùng GridSearchCY dò n_estimators, ta có thể dùng Dừng sớm qua n_iter_no_change:

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2,
    learning_rate=0.05,
    n_estimators=500,
    n_iter_no_change=10, # Dùng nếu 10 lan lạp lien tiep khong giảm lơi
    random_state=42
)
gbrt_best.fit(X, y)

print(gbrt_best.n_estimators_) # Chỉ cần 92 cay (tư dung som) thay vi 500
```

Gemini Notebook

Tăng cường Gradient dựa trên Biểu đồ (HGB)

GBRT chậm trên dữ liệu lớn. Scikit-Learn cung cấp Histogram-Based Gradient Boosting (HGB) tối ưu cho dữ liệu hàng triệu dòng.



Cơ chế: Nhóm các đặc trưng lại (binning) thành tối đa 255 nhóm (bins). Giảm đáng kể số ngưỡng phân chia.



Độ phức tạp: Giảm từ O(b×m× log(m)) xuống O(b×m), huấn luyện nhanh hơn hàng trăm lần.



Xử lý dữ liệu: HGB xử lý luôn được cả Missing values và Categorical Features (không cần Imputer hay OneHotEncoder).

Gemini Notebook

Mã nguồn: HistGradientBoostingRegressor

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
    remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)
hgb_reg.fit(housing, housing_labels)
```

Gemini Notebook

Phân lớp xếp chồng (Stacking)

Stacking (Stacked Generalization): Thay vì dùng hàm tầm thường như Bỏ phiếu đa số, tại sao không huấn luyện một mô hình máy học khác để tổng hợp các dự đoán?

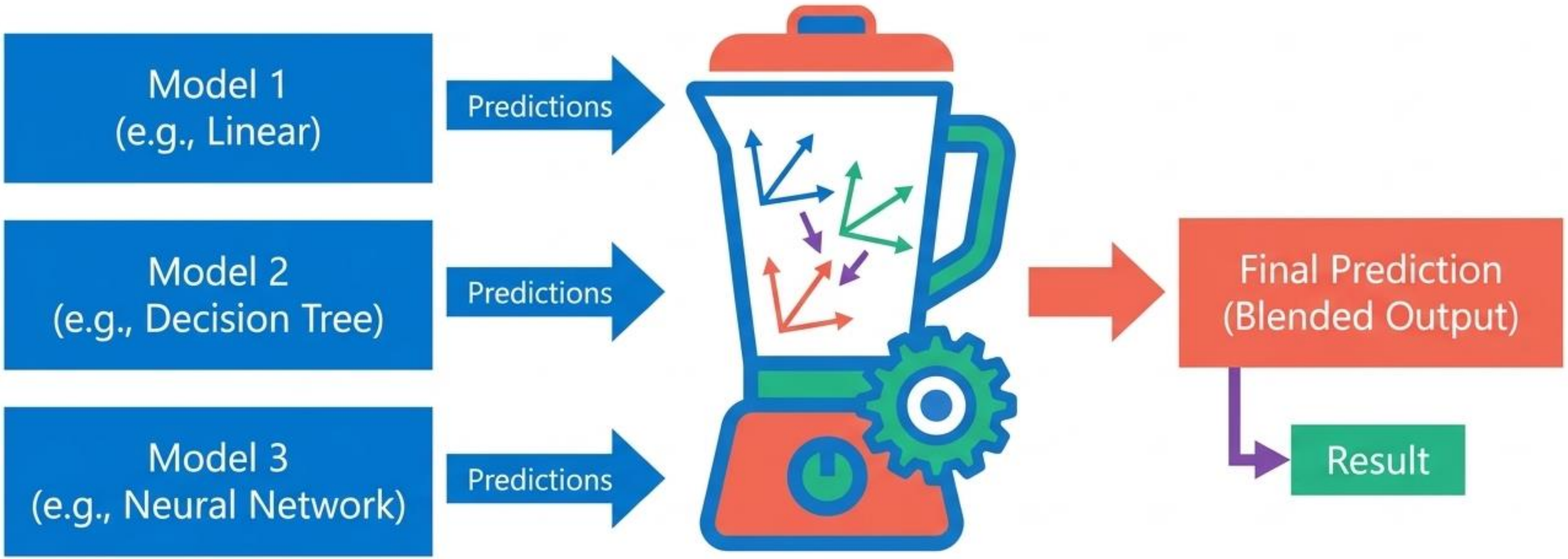
Bộ mô hình lớp cuối cùng được gọi là bộ trộn (blender) hay meta-learner.

Lấy các dự đoán của các bộ học cơ sở làm đầu vào (input) và học cách tạo ra dự đoán chính xác nhất từ đó.



Gemini Notebook

Tổng hợp dự đoán bằng bộ trộn



Hình 7-11. Sử dụng bộ trộn để kết hợp 3 mô hình

Gemini Notebook

Khái niệm bộ trộn (Blender)

Để huấn luyện bộ trộn, ta chia dữ liệu thành 2 phần:



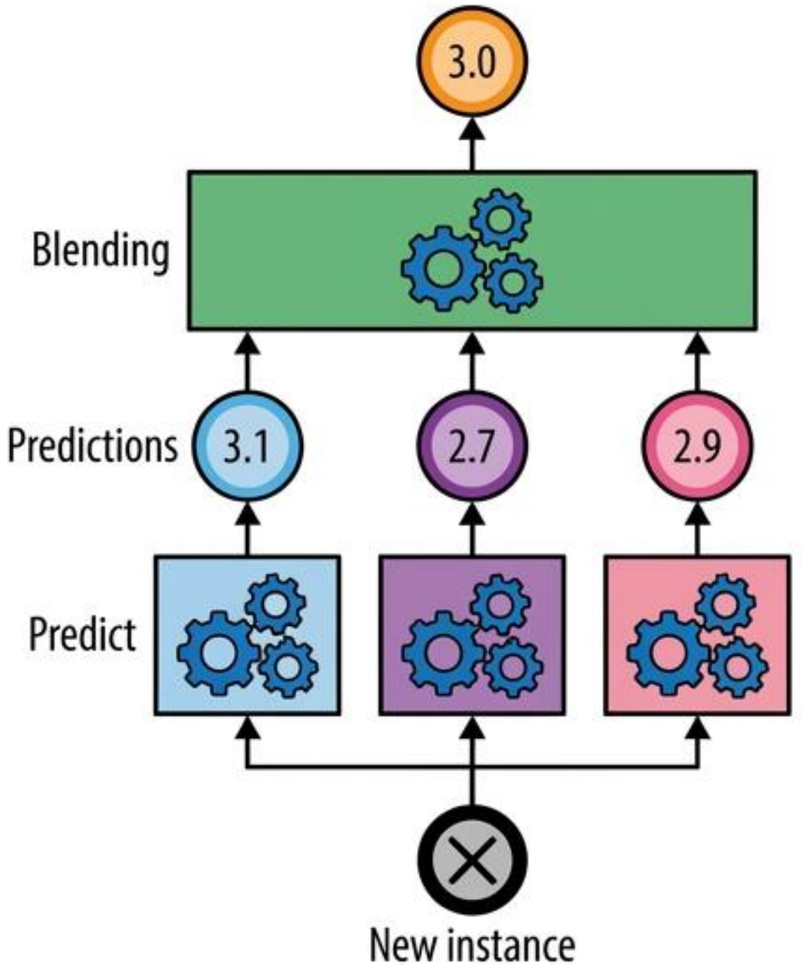
Tập 1 (Tập huấn luyện): Dùng để huấn luyện các mô hình cơ sở.



Tập 2 (Tập pha trộn - Blend/Hold-out): Các mô hình cơ sở sẽ dự đoán trên tập này. Các dự đoán xuất ra sẽ đóng vai trò là đặc trưng (features) mới. Bộ trộn sẽ được huấn luyện trên tập đặc trưng mới này kết hợp với nhãn gốc. **Scikit-Learn** sử dụng cross_val_predict (K-Fold) để tránh làm lãng phí dữ liệu.

Gemini Notebook

Huấn luyện bộ trộn trong tập hợp



Hình 7-12. Tạo dữ liệu huấn luyện cho bộ trộn qua Out-of-Sample Predictions

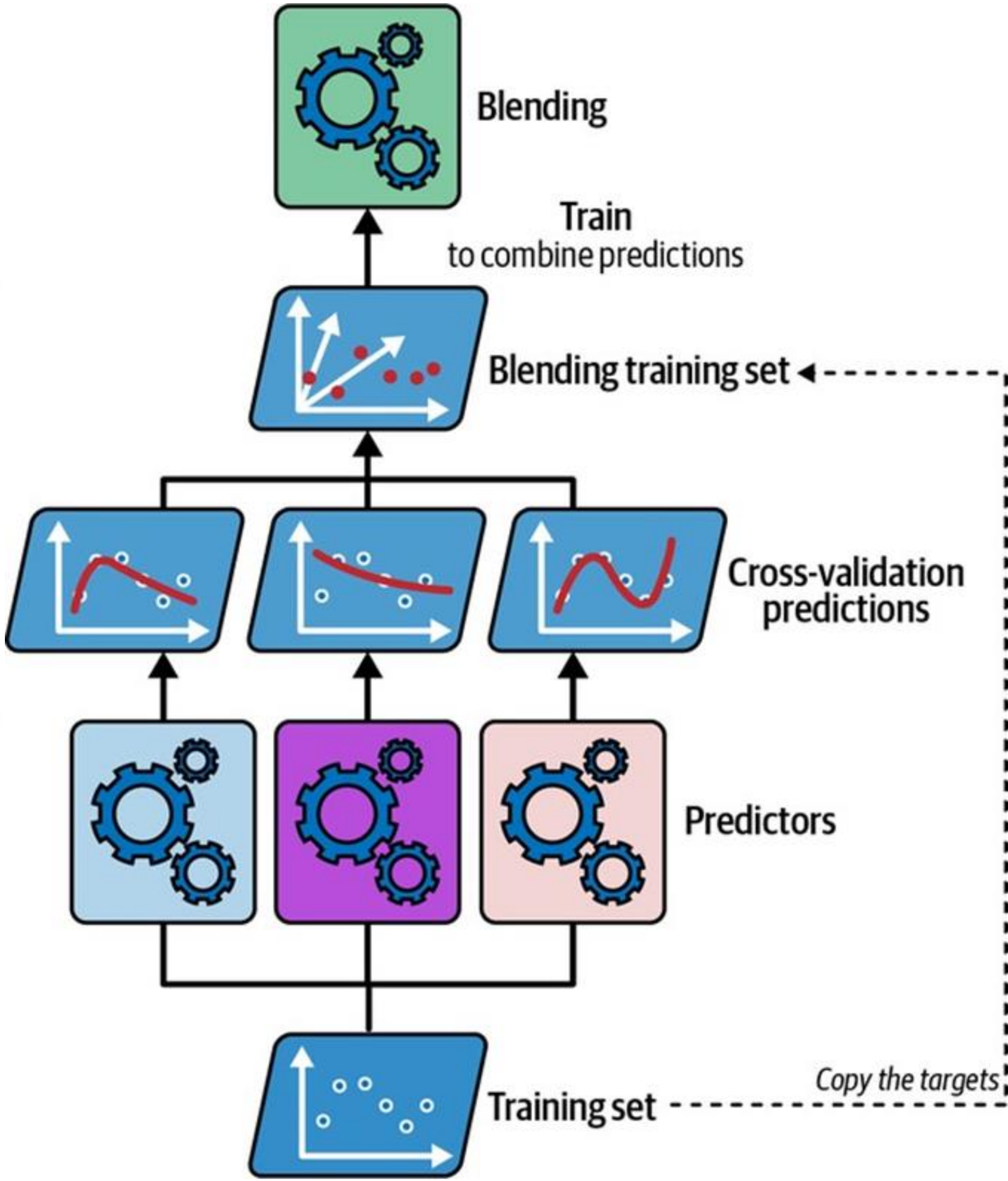
Gemini Notebook

Kiến trúc xếp chồng phức tạp

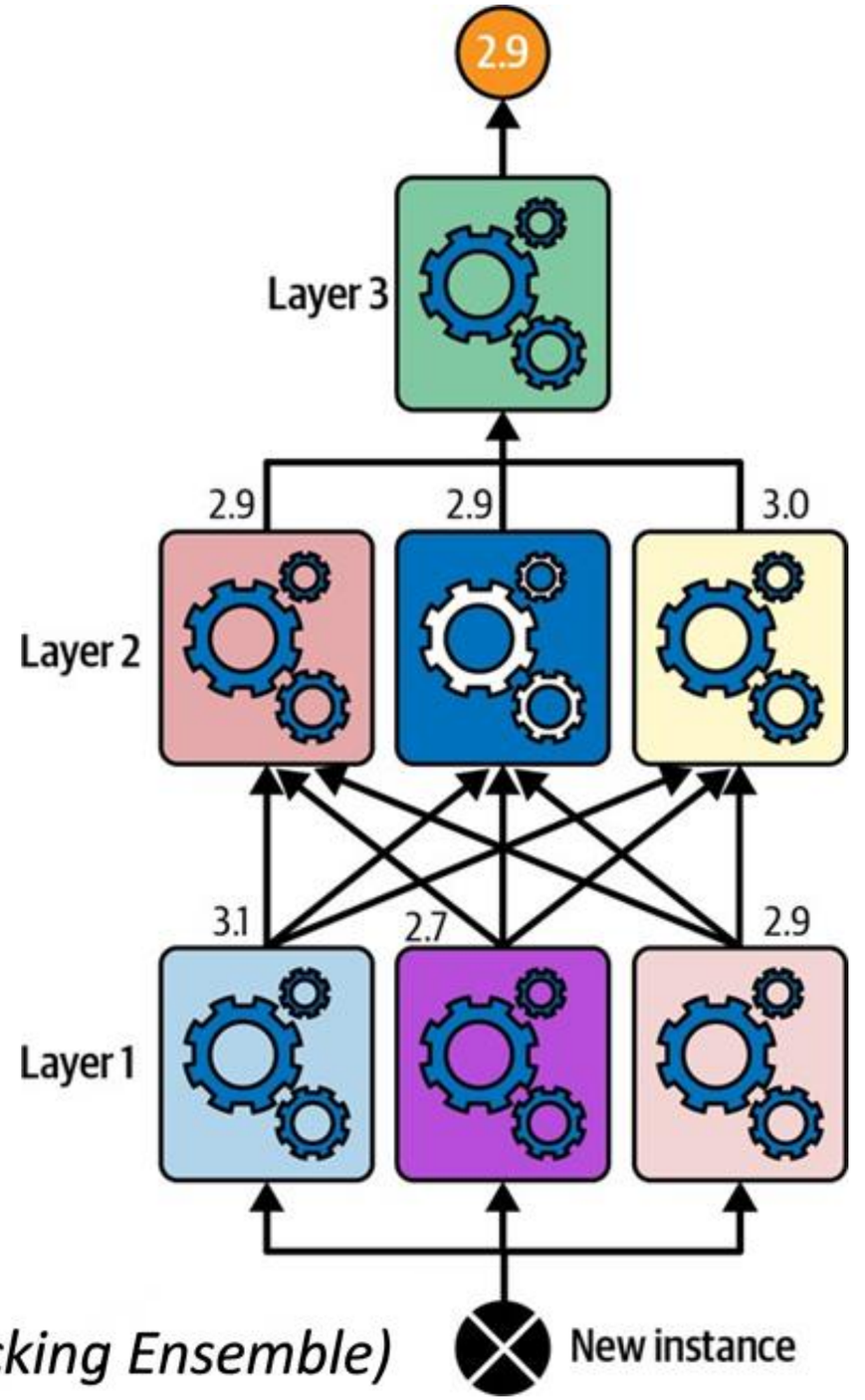
Không dừng lại ở 1 bộ trộn, ta có thể huấn luyện nhiều bộ trộn khác nhau (ví dụ: Linear, Random Forest) trên cùng một layer.

Sau đó thêm 1 bộ trộn Tối cao lên trên cùng (Layer 3) để đưa ra dự đoán cuối.

Càng nhiều lớp, mô hình càng tốn kém thời gian huấn luyện và độ phức tạp tính toán.



Tập hợp xếp chồng đa lớp



Hình 7-13. Mạng Stacking đa tầng (Multi-layer Stacking Ensemble)

Cài đặt StackingClassifier trong Scikit-Learn



```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # 5-Fold cross validation
)
stacking_clf.fit(X_train, y_train)
```

Gemini Notebook

Tổng kết Chương 7

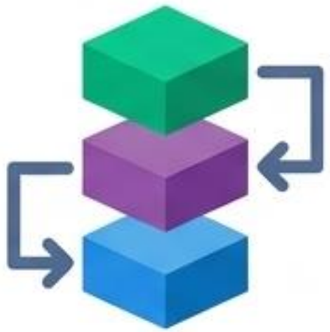
Học tổ hợp (Ensemble Learning) là công cụ quyền lực đẩy hiệu năng mô hình đến giới hạn.



Voting & Bagging/Pasting: Đơn giản, dễ song song hóa, giảm phương sai cực tốt. Rừng ngẫu nhiên (Random Forest) là thuật toán tiêu biểu, hiệu năng ổn định.



Boosting (AdaBoost, GBRT, HGB): Tập trung sửa sai, chuyển bộ học yếu thành mạnh. HGB là đột phá về tốc độ cho dữ liệu lớn.



Stacking: Đẩy mạnh sức mạnh thông qua mô hình Meta-learner thay vì trung bình cứng nhắc.



Rừng ngẫu nhiên và GBRT luôn là những mô hình đầu tiên nên thử nghiệm trong mọi dự án Machine Learning dạng bảng.

Gemini Notebook

Hỏi & Đáp

CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE!

Q & A

Gemini Notebook